



Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Faculty of Computer Science and Engineering

CS424 L-Lab Compiler Construction

Lab 10: Advanced Target Code Generation and Instruction Mapping

1. Comprehensive Theory

1.1 The Role of the Back-End

Target Code Generation is the final phase of the compilation process. While the front-end focuses on the *meaning* of the source code, the back-end focuses on the *execution* on physical hardware. The primary goal is to map the Intermediate Representation (IR) to the instruction set architecture (ISA) of the target processor.

1.2 Key Challenges in Code Generation

1. **Instruction Selection:** A single IR operation might be representable by multiple assembly instructions. The generator must choose the most efficient one.
2. **Register Allocation:** CPU registers are the fastest memory locations. Since we have a limited number (e.g., R0-R15), the compiler must decide which variables stay in registers and which are "spilled" back to slower RAM.
3. **Instruction Ordering:** The order of instructions can affect performance due to CPU pipelining and data dependencies.

1.3 The Load-Store Architecture Logic

In most RISC-based or modern architectural simulations, the CPU cannot perform arithmetic directly on memory.

- **Incorrect:** ADD memory_address_A, memory_address_B
- **Correct (The 3-Step Cycle):**
 1. **Load:** Fetch data from memory into a register.
 2. **Operate:** Perform the math (ADD, SUB, etc.) using the register.
 3. **Store:** Move the result from the register back to a memory address.

2. Detailed Implementation: The Code Generator

We will build a C program that simulates this back-end. It treats each line of Three-Address Code (TAC) as a instruction to be converted into Assembly.

A. Input Format (input.txt)

The program expects a four-column format: Result, Arg1, Operator, Arg2.

```
t1 10 + 20  
t2 t1 * 5  
res t2 = NULL
```

B. The Generator Code (codegen.c)

```
#include <stdio.h>  
#include <string.h>  
#include <stdlib.h>  
  
/**  
 * Function: translate_instruction  
 * -----  
 * This function handles the "Instruction Selection" logic.  
 * It maps TAC symbols (+, -, *, /) to Assembly Mnemonics (ADD, SUB, MUL,  
 * DIV).  
 */  
void translate_instruction(char* result, char* arg1, char* op, char* arg2) {  
    // We use a single register R0 for this basic demonstration.  
    // In a real compiler, a 'Register Descriptor' would track available registers.  
  
    printf("\n/* --- Generating Code for: %s = %s %s %s --- */\n", result, arg1,  
    op, arg2);
```

```
// STEP 1: LOAD
```

```
// Bring the first operand from memory into the CPU register
```

```
printf("LOAD R0, %s\n", arg1);
```

```
// STEP 2: OPERATE
```

```
// Select the assembly instruction based on the TAC operator
```

```
if (strcmp(op, "+") == 0) {
```

```
    printf("ADD R0, %s\n", arg2); // Result in R0 = R0 + arg2
```

```
}
```

```
else if (strcmp(op, "-") == 0) {
```

```
    printf("SUB R0, %s\n", arg2); // Result in R0 = R0 - arg2
```

```
}
```

```
else if (strcmp(op, "*") == 0) {
```

```
    printf("MUL R0, %s\n", arg2); // Result in R0 = R0 * arg2
```

```
}
```

```
else if (strcmp(op, "/") == 0) {
```

```
    printf("DIV R0, %s\n", arg2); // Result in R0 = R0 / arg2
```

```
}
```

```
else if (strcmp(op, "=") == 0) {
```

```
    // Direct assignment: No math operation needed.
```

```
    // R0 already contains the value of arg1.
```

```
}
```

```
// STEP 3: STORE
```

```
// Move the calculated value from the register back to the result variable in  
memory
```

```

        printf("STORE %s, R0\n", result);
    }

int main() {
    char result[20], arg1[20], op[10], arg2[20];

    // Open the intermediate code file
    FILE *file = fopen("input.txt", "r");

    if (file == NULL) {
        printf("Error: input.txt not found. Please create the file first.\n");
        return 1;
    }

    printf("; #####\n");
    printf("; # AUTOMATED TARGET CODE GENERATOR   #\n");
    printf("; #####\n");

    /* Logic: Read four strings per line until End Of File (EOF).
       Format: result operand1 operator operand2
    */
    while (fscanf(file, "%s %s %s %s", result, arg1, op, arg2) != EOF) {
        translate_instruction(result, arg1, op, arg2);
    }

    fclose(file);
}

```

```
printf("\n; Code generation complete.\n");  
return 0;  
}
```

3. Execution and Testing Procedure

1. **Preparation:** Open a text editor and save the code as codegen.c.
2. **Input Data:** Create input.txt in the same folder. Add lines of TAC, ensuring you use four parts per line (use NULL for the 4th part in assignments).
3. **Compilation:**

```
gcc codegen.c -o generator
```

4. **Running:**

```
./generator
```

5. **Verification:** Compare the output with the mathematical logic. For t1 5 + 10, you should see:
 1. LOAD R0, 5
 2. ADD R0, 10
 3. STORE t1, R0

4. Lab Tasks

Task 1: Relational Logic

Extend the generator to support comparison operators (<, >, ==). Use the assembly mnemonic CMP and JMP (Jump) to simulate how a compiler handles branch instructions.

Task 2: Register Tracking Optimization

A common inefficiency is re-loading a value into a register that is already there.

- **The Problem:** If t1 is stored in the first line and then used as arg1 in the second line, your current code generates STORE t1, R0 followed by LOAD R0, t1.

- **The Task:** Modify the program to remember what is currently in R0. If the next instruction needs that same value, skip the LOAD instruction.

Task 3: Multiple Registers

Modify the code to use two registers, R0 and R1. Load arg1 into R0 and arg2 into R1 before performing the math. Discuss how this changes the "Store" step.